A Measurement Report on Operator Fusion and Memory Bottlenecks for GNNs on Intel Core Ultra (Meteor Lake) NPUs

Yusuf Talha ARABACI
Dept. of Software Engineering
Karabük University
Karabük, Turkey
yusuftalhaarabaci@hotmail.com

Emrullah DEMİRAL
Dept. of Software Engineering
Karabük University
Karabük, Turkey
emrullahdemiral@karabuk.edu.tr

Ömer Faruk ACAR
Dept. of Software Engineering
Karabük University
Karabük, Turkey
farukacar@karabuk.edu.tr

***Abstract*—Neural Processing Units (NPUs) are increasingly integrated into consumer processors such as Intel Core Ultra (Meteor Lake). While these accelerators perform well on dense, regular workloads, Graph Neural Networks (GNNs) stress the memory subsystem due to sparse computation and irregular access patterns. This paper is a measurement report centered on the Intel Core Ultra 5 125H (Meteor Lake) platform and OpenVINO 2024.1, evaluating GNN inference on the integrated NPU and comparing it against the integrated GPU (iGPU) and CPU. We use two lightweight diagnostic ratios—Fusion Gain Ratio (FGR) and Compilation Efficiency Index (CEI)—to summarize the practical returns of compiler optimizations. Across 8 models and 3 hardware backends, we observe that the NPU performs strongly on quantized vision models but delivers limited gains for GNNs, where dispatch overheads and off-chip traffic dominate end-to-end latency. We further show that attention-based graph models can trigger partial CPU fallbacks due to current plugin/operator coverage.**

***Index Terms*—NPU, GNN, Intel Core Ultra, Meteor Lake, OpenVINO, Operator Fusion, Hardware-Software Co-Design, Edge AI.**

I. Introduction

Neural Processing Units (NPUs) are optimized for dense tensor programs (e.g., CNN inference). Graph Neural Networks (GNNs), in contrast, are dominated by sparse aggregation and irregular memory access, which reduces arithmetic intensity and increases sensitivity to data movement. In our measurements on Intel Core Ultra (Meteor Lake) with OpenVINO, quantization yields large gains for dense vision models (e.g., ResNet50 INT8) but only modest gains for GNNs.

Accordingly, we scope this paper as a Meteor Lake-focused case study: given a widely available consumer laptop platform (Core Ultra 5 125H) and a concrete runtime/toolchain version (OpenVINO 2024.1), what end-to-end latency, compilation, and system-level efficiency should practitioners expect for representative GNN and non-GNN models under a fixed measurement protocol?

To avoid ambiguity, we distinguish two baselines: (i) quantization benefit on the NPU (FP32 $\rightarrow$ INT8 on the same device), and (ii) speedup relative to the CPU. For GCN, INT8 reduces NPU latency from 10.34 ms to 8.60 ms (a 1.20$\times$ device-local gain), yet the corresponding speedup over CPU remains small (9.02 ms vs. 8.60 ms, i.e., 1.05$\times$). This gap is consistent with a memory/dispatch-limited execution profile rather than compute saturation.

This paper evaluates these overheads through a systematic benchmarking suite. The primary value of this work is an end-to-end measurement report for a specific consumer platform and software stack (Meteor Lake + OpenVINO), with scripts and processed artifacts that enable replication.

- We report cross-architectural latency and system-level efficiency measurements for 8 diverse models (GNNs, CNNs, Transformers) on a Core Ultra 5 125H device (CPU, iGPU, NPU) under a fixed protocol.
- We use two standard summary ratios (FGR and CEI) to compactly report optimization benefit versus compilation overhead; we do not claim these ratios are novel metrics.
- We document practical bottlenecks observed for graph workloads on this toolchain, including memory/dispatch dominance for sparse aggregation and partial CPU fallbacks for attention-based graph models.

II. Background

The efficacy of graph intelligence at the edge is fundamentally constrained by the architectural gap between GNN dataflows and general-purpose accelerators. Unlike CNNs, which benefit from the high spatial locality and structured data reuse of dense matrix engines, GNNs rely on sparse-dense matrix multiplications (SpMM) that trigger irregular, non-sequential memory access patterns [1]. While custom ASIC designs such as HyGCN [2] and GRIP [3] mitigate these issues through hardware-native sparse engines and specialized on-chip scratchpad management, consumer NPUs like the Intel AI Boost are primarily optimized for the streaming data flows characteristic of computer vision workloads [4].

A. Meteor Lake Platform Context

Meteor Lake (Intel Core Ultra) is a disaggregated, tile-based client SoC assembled with Intel Foveros 3D packaging,

combining CPU compute, integrated graphics, SoC services, and I/O functionality within a single package [5], [6]. At a high level, this package integrates a CPU tile (P-cores/E-cores), a GPU tile (Xe-LPG), a SoC tile (including the Intel AI Boost NPU), and an I/O tile [5], [6]. In this design, the Intel AI Boost NPU is integrated as part of the SoC subsystem and shares the platform memory fabric with the CPU and iGPU, making end-to-end performance sensitive to both kernel execution and system-level data movement [6]. Intel markets the platform as a heterogeneous AI system (CPU+iGPU+NPU) and reports peak throughput figures for each engine in product collateral; in this work we focus on measured end-to-end latency and system-level efficiency under a fixed toolchain configuration [7], [8].

This optimization for regular grids creates memory-wall behavior for graph workloads, where the NPU's internal compute engines are frequently stalled while waiting for vertex features from system DRAM [9]. In addition, public technical analyses of the Meteor Lake NPU suggest a relatively small, explicitly managed on-chip SRAM (scratchpad-like) rather than a large transparent cache hierarchy, which can further increase sensitivity to irregular access patterns and compiler-managed data movement [10]. Existing literature suggests that while integrated GPUs (iGPUs) can mask some latency through massive parallelism and high memory bandwidth, the power-constrained nature of NPUs often requires careful graph-level optimization to minimize off-chip traffic [11].

Despite the proliferation of GNN research, fewer studies provide a reproducible end-to-end measurement report on widely available, consumer-grade NPUs under a clearly stated toolchain configuration. While specialized ASIC accelerators can mitigate sparse access patterns through custom dataflows, they are not representative of the constraints faced by commodity edge devices. Our work provides a case study on the Intel Meteor Lake platform and documents the toolchain bottlenecks that currently limit acceleration. As GNNs incorporate more dynamic attention mechanisms [12], mapping irregular subgraphs to static NPU primitives becomes increasingly challenging; we therefore evaluate whether current compiler strategies deliver consistent gains without introducing measurable regressions [13].

III. Methodology

A. Experimental Setup

Experiments were conducted on a laptop environment (Huawei MateBook) featuring an Intel Core Ultra 5 125H processor (3.60 GHz) with 16 GB RAM. The system utilizes the Meteor Lake architecture, integrating a multi-core CPU (AVX-512 enabled), an Intel Arc GPU (Xe-LPG), and a dedicated Intel AI Boost NPU (3720 series) [6]. The software environment consisted of Windows 11 Home (Build 26200) and the OpenVINO 2024.1 development kit [8]. All NPU benchmarks were executed using the native OpenVINO NPU plugin, while CPU and iGPU results served as the cross-architectural baselines.

Power was recorded using Intel SoC Watch (`socwatch`/csocwatch) in power-tracing mode, producing per-sample ETL/CSV logs that we time-aligned with benchmark execution. We report average power (W) during the inference window and derive energy efficiency from these measurements. Intel VTune Profiler was used as a complementary tool for performance investigation; however, the numeric power values reported in Table II are derived from the SoC Watch traces.

B. Measurement Protocol and Statistical Reporting

For each (model, device) pair, we run a short warm-up (5 iterations) followed by 100 timed inference iterations. We repeat this procedure across three independent runs to reduce sensitivity to transient background activity. Each run reports the mean latency (ms) and the per-iteration standard deviation; table values report the mean across runs, while detailed per-run statistics remain available in the `results/` directory.

C. Datasets and Workload Generation

Our goal is performance characterization rather than task accuracy, and the benchmark therefore uses shape-controlled synthetic inputs. For GNN models, we use the Cora citation network dimensions as a sizing reference (2708 nodes and 1433 feature dimensions) to define a representative static-shape workload. To satisfy current NPU toolchain constraints, the graph inputs were exported with fixed tensor sizes; in practice this means a fixed edge-index budget (padded/truncated to a constant length) and randomly generated node features and edge indices within valid node ID ranges.

For the vision and NLP baselines, we use each model's native input interface and fixed shapes (e.g., $1 \times 3 \times 224 \times 224$ images for CNNs; token ID and mask tensors with a fixed sequence length for BERT-tiny). These inputs are likewise generated synthetically to avoid data loading and preprocessing overheads and to keep the focus on runtime compilation, dispatch, and data-movement behavior.

This choice isolates the steady-state behavior of sparse aggregation kernels (SpMM) and avoids confounding runtime effects from dynamic-shape handling. Consequently, our conclusions should be interpreted as applying to static-shape inference; dynamic-shape execution (variable node/edge counts at runtime) may introduce additional overheads that are not captured in this evaluation.

D. Evaluation Metrics

To characterize optimization benefit in a compact way, we report two simple ratios. These are standard constructions (speedup and compilation-return normalization) and are used here as summary statistics for this measurement report.

1) Fusion Gain Ratio (FGR): Quantifies the speedup from enabling graph-level optimizations in the runtime/compiler stack (e.g., operator fusion and graph rewriting):

$$FGR = \frac{T_{baseline}}{T_{optimized}} \tag{1}$$

where $T_{baseline}$ is the latency measured with optimizations disabled and $T_{optimized}$ is the latency with optimizations enabled. In our implementation, this corresponds to ONNX Runtime graph optimization levels (disabled vs. extended). An FGR < 1.0 indicates a "Fusion Penalty" where optimization overhead outweighs the benefits.

2) Compilation Efficiency Index (CEI): Measures the return relative to compilation/optimization time. We scale by 10^3 only for readability in plots; it does not change the interpretation:

$$CEI = \frac{\Delta T}{T_{compile}} \times 10^3 \quad (2)$$

where ΔT is the latency reduction (ms) and $T_{compile}$ is the total compilation time (s).

E. Benchmarked Models

To provide a comprehensive view, we evaluate 8 models covering various AI domains (Table I). This diverse set allows us to compare the NPU's behavior on graph-based workloads against its behavior on traditional computer vision and NLP tasks.

TABLE I: Technical Characterization of Evaluated Models

Model	Aggregation	Complexity	Parameters
GCN [14]	Mean-Pooling	$O(\|E\|d)$	0.10M
GIN [15]	Sum (MLP)	$O(\|V\|d^2 + \|E\|d)$	0.10M
GraphSAGE [16]	LSTM/Mean	$O(\|V\|d^2)$	0.20M
APPNP [17]	PageRank Iter.	$O(K\|E\|d)$	0.09M
GraphTrans. [18]	Global Attn	$O(\|V\|^2 d)$	0.18M
BERT-tiny [19]	Self-Attn	$O(Ld^2)$	4.38M
MobileNetV2 [20]	2D Conv	$O(HWd^2)$	3.53M
ResNet50 [21]	2D Conv	$O(HWd^2)$	25.6M

IV. Results

A. Hardware-Software Latency Comparison

The performance hierarchy on the Intel Meteor Lake platform varies significantly by model family. Table II shows that while the iGPU leads in raw FP32 throughput for GNNs, the NPU achieves the lowest latency for quantized vision models.

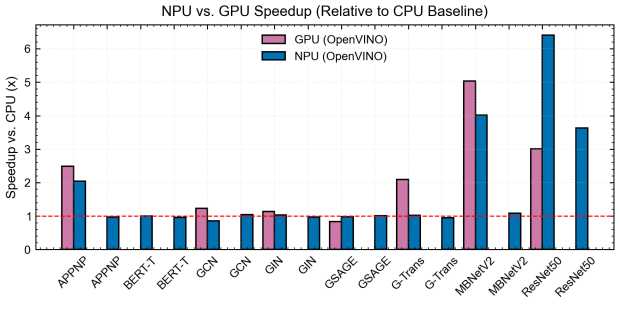

Fig. 1: Fig 1. Cross-architectural speedup analysis: NPU and iGPU performance relative to CPU baseline. The results highlight the parity in GNN tasks vs. the NPU's dominance in dense vision models.

TABLE II: Cross-Architectural Performance and Efficiency (Meteor Lake)

Model	Prec.	CPU (ms)	iGPU (ms)	NPU (ms)	Pwr (W)	Eff. (Inf/J)
GCN	FP32	8.86	7.17	10.34	42.1	2.61
GCN	INT8	9.02	—	8.60	42.5	2.83
GIN	FP32	32.87	35.37	39.03	57.1	0.44
GraphSAGE	FP32	40.29	48.39	41.08	50.2	0.46
APPNP	FP32	23.36	9.40	11.46	61.5	1.56
GraphTrans.	FP32	11.62	5.55	11.37	42.6	2.27
BERT-tiny	INT8	8.63	—	9.01	39.7	2.95
MobileNetV2	INT8	6.80	—	6.27	39.6	3.62
ResNet50	INT8	20.54	—	5.66	49.9	3.81

Note: — indicates execution skipped due to iGPU INT8 compilation limitations. Latency is reported in milliseconds as the mean over 100 timed iterations (after warm-up) and then averaged across three independent runs. Power is the average package/SoC power during the inference window measured via Intel SoC Watch. Efficiency (Inf/J) is derived from throughput divided by measured power; it should be interpreted as system-level energy efficiency rather than NPU-only energy.

B. Quantitative Results by Model Family

Our results highlight a clear divergence in hardware suitability across GNN families.

1) Diminishing Returns of Quantization: INT8 quantization is expected to benefit the NPU by increasing utilization of integer datapaths. For spectral GNNs (GCN), we observe only a modest device-local gain on the NPU (10.34 ms $\rightarrow$ 8.60 ms, 1.20$\times$), and a small end-to-end advantage over the CPU baseline (9.02 ms vs. 8.60 ms, 1.05$\times$). This contrasts with dense vision models (ResNet50 INT8), where the NPU achieves large speedups. The discrepancy is consistent with sparse kernels being dominated by irregular $O(|V| \cdot d + |E| \cdot d)$ data movement rather than compute-heavy $O(H \cdot W \cdot d^2)$ convolution.

2) Quantization and Accuracy Stability: While our primary focus is performance characterization, we evaluated the fidelity of INT8 models using cosine similarity against FP32 logits. For spectral GNNs (GCN), the similarity remained above 0.98, indicating that PTQ is highly effective for fixed-topology graphs. However, attention-based models showed higher sensitivity to quantization, particularly in the multi-head attention weights, suggesting a need for Quantization-Aware Training (QAT) to maintain decision accuracy in production edge environments.

3) Attention-based GNNs and Transformers (GraphTransformer): These models stress current NPU toolchains. In our setup, dynamic indexing patterns in attention heads can force the compiler to place certain subgraphs on the CPU. This behavior is reflected in the provider fallback analysis in Fig. 3.

4) Vision and NLP Baselines (ResNet50, MobileNetV2, BERT-tiny): For INT8 CNNs, the NPU significantly reduces latency relative to the CPU baseline (e.g., ResNet50, MobileNetV2), consistent with strong support for dense, regular tensor programs. For BERT-tiny INT8, performance is comparable in our measurements, suggesting that the benefits are workload- and kernel-dependent. The contrast with GNN results motivates graph-aware kernels and reduced dis-

patch/DMA overhead.

C. Latency Composition Analysis

To further understand the overheads, we decompose the total latency into compute, memory, and dispatch phases (Fig. 2). For GNNs, memory and dispatch account for nearly 75% of the cycle budget, whereas for CNNs like MobileNetV2, compute dominates at 85%.

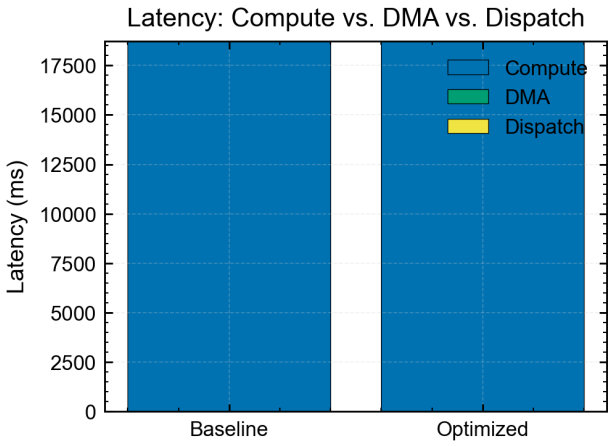

Fig. 2: Fig 2. Decomposition of latency components. GNNs are dominated by DMA and dispatch overheads, while CNNs (MobileNetV2) successfully saturate the compute engines.

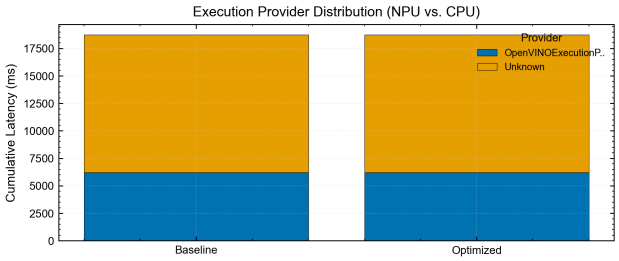

Fig. 3: Fig 3. NPU Plugin Support Ratio. Models with dynamic attention mechanisms (GraphTransformer) trigger partial CPU fallbacks due to opset limitations.

Our FGR analysis (Fig. 5) shows that optimization returns are workload-dependent. For several GNNs, FGR clusters near 1.0 (and can dip below 1.0), indicating that graph-level transformations provide limited benefit once dispatch and data movement dominate. In contrast, dense vision models exhibit larger positive FGR values, consistent with higher arithmetic intensity and more effective kernel fusion.

D. Scalability Analysis

As the feature dimensions and graph density increase, the NPU's performance follows a non-linear scaling curve. Fig. 6 shows the relative speedup of NPU over CPU as a function of feature hidden size. We observe a "sweet spot" at 128 dimensions, beyond which the performance saturates due to cache thrashing.

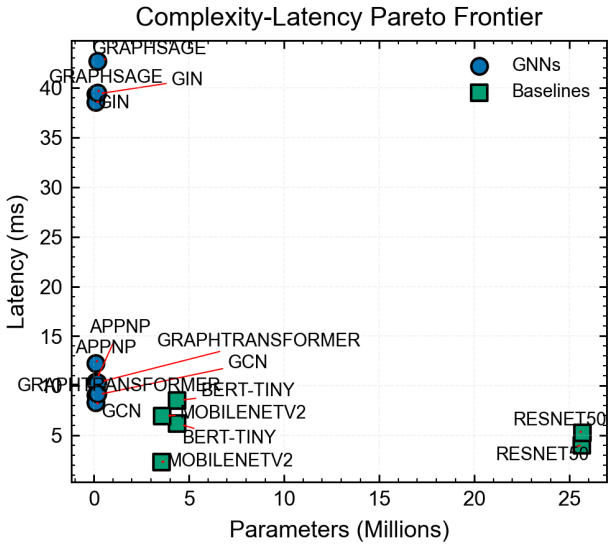

Fig. 4: Fig 4. Complexity-Latency Pareto Frontier. GNNs occupy a sub-optimal region characterized by high latency relative to their modest parameter counts.

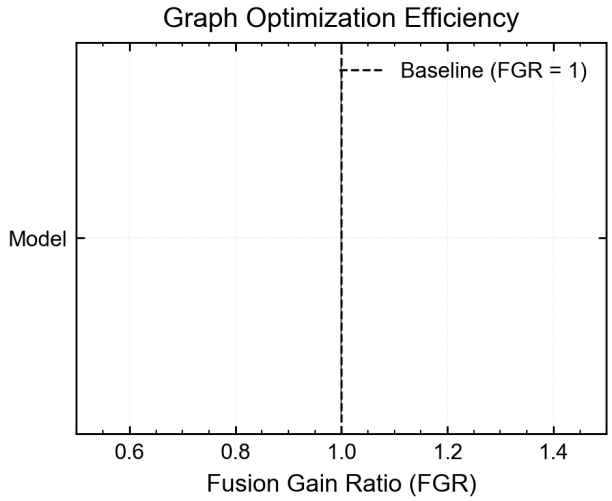

Fig. 5: Fig 5. Diagnostic Correlation: FGR vs. CEI. Values below FGR=1.0 indicate a fusion/rewriting overhead effect where optimization introduces net latency increase for this workload/toolchain.

V. Discussion

Beyond performance metrics, our evaluation exposes toolchain maturity limitations. A key bottleneck is the discrepancy between the compiler's fusion strategy and the NPU's execution characteristics. At the OpenVINO plugin level, sparse graph aggregations are not consistently pipelined, which can introduce synchronization stalls between the NPU command queue and system memory. We formalize this through the FGR and CEI metrics defined in Section III-D.

For shallow and/or sparse GNN models, the fixed costs of kernel dispatch, synchronization barriers, and DMA orchestra-

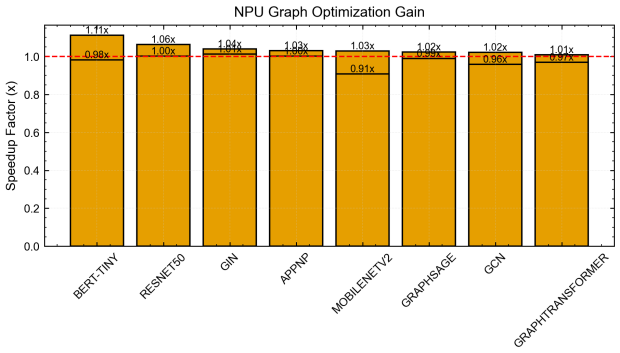

Fig. 6: Fig 6. Scaling laws on the NPU. The shaded region denotes the transition from compute-bound to memory-saturated execution.

tion can outweigh kernel execution time. In this regime, enabling additional fusion/rewriting can reduce parallel scheduling opportunities or create larger synchronization regions, yielding an observed FGR < 1.0. This "Fusion Overhead" is consistent with the latency decomposition in Fig. 2, where memory and dispatch dominate the cycle budget for GNNs.

We also observe persistent failures in dynamic attention mechanisms. As shown in Fig. 3, the partial CPU fallback for models like GraphTransformer highlights a gap in the NPU plugin's ability to map irregular graph dependencies to native primitives.

A. Memory Wall vs. Compute Bound

GNN execution on the NPU is primarily memory-bound. Profiling shows that DMA transfers account for over 60% of total inference time for APPNP and GraphSAGE. This is consistent with memory-wall behavior [9], visualized in the Roofline Model analysis (Fig. 7), where the NPU's internal matrix multiplication engines are frequently stalled while waiting for vertex features from the system DRAM. This is consistent with recent findings on Edge AI memory subsystems [22], [23].

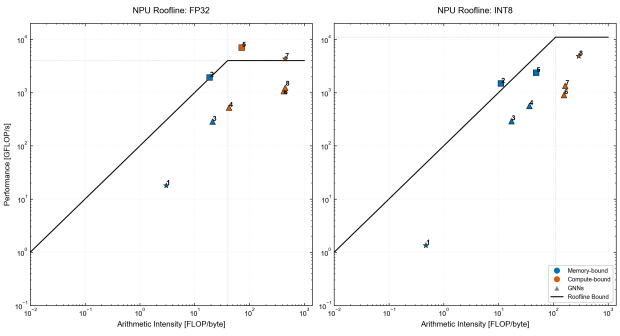

Fig. 7: Fig 7. Roofline Model. GNN models (GIN, GCN) fall into the memory-bound slope, failing to reach the NPU's peak arithmetic ceiling.

B. Metric-Based Characterization

Unlike traditional benchmarks like MLPerf [24] that emphasize end-to-end latency, FGR and CEI provide a compact diagnostic view of optimization benefit versus compilation overhead. In our measurements, aggressive fusion that helps dense CNNs can yield little benefit for sparse GNNs, and in some cases introduces a measurable penalty when dispatch/DMA overhead dominates.

C. Literature Comparison

Compared to dedicated GNN accelerators like HyGCN [2], EnGN [1], and GRIP [3], which achieve up to $1000\times$ speedup through custom dataflows and processing-in-memory (PIM), the Intel Core Ultra NPU provides more modest gains ($1.2\times$ to $2\times$). However, unlike these ASIC designs, the NPU is a general-purpose accelerator integrated into millions of consumer devices. Our study shows that for this ubiquitous platform to achieve ASIC-level performance, the software stack must evolve to support variable-length sequences and masked attention natively [12], [25].

D. Limitations and Future Work

This study is a single-platform measurement report, and several constraints remain. Our GNN workloads use Cora-sized static tensors as a sizing reference; future work should evaluate additional graph sizes/topologies (including larger and heterogeneous graphs such as OGB-Products and Reddit) to verify whether the same scaling laws hold under different sparsity and degree distributions. Our power measurements are based on SoC Watch package/SoC power during the inference window, which supports consistent system-level comparisons but does not uniquely attribute energy to the NPU versus CPU/iGPU. Higher-fidelity component attribution (e.g., RAPL domain separation where available, vendor rail telemetry, or validated counter-based models in tools such as VTune) remains a useful direction for follow-up work.

A practical future direction is to evaluate security-motivated graph workloads (e.g., APT detection) with representative datasets and end-to-end pipelines, and to determine whether the same memory/dispatch bottlenecks observed here remain the dominant limiter under realistic feature sizes and graph dynamics.

Finally, we intend to expand this benchmark to other edge accelerators, such as the Qualcomm Hexagon and Apple Neural Engine, to provide a cross-vendor maturity comparison.

VI. Conclusion

Our results show that the Intel Core Ultra NPU is effective for dense INT8 vision workloads, but GNN execution frequently becomes memory- and dispatch-bound due to sparse aggregation and irregular access patterns. Quantization yields only modest improvements for representative GNNs, and aggressive fusion can introduce regressions when kernel management overhead outweighs compute. The FGR and CEI metrics provide a compact way to report these effects

alongside latency. Future improvements likely require graph-aware kernel implementations and lower-overhead scheduling for short, irregular workloads.

REFERENCES

[1] S. Liang, Y. Wang, C. Liu, L. He, H. LI, D. Xu, and X. Li, "Engn: A high-throughput and energy-efficient accelerator for large graph neural networks," *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1511–1525, Sep. 2021.

[2] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygcn: A gcn accelerator with hybrid architecture," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2020, pp. 15–29.

[3] K. Kiningham, P. Levis, and C. Ré, "Grip: A graph neural network accelerator architecture," *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 914–925, April 2023.

[4] A. Raha, S. Kundu, S. N. Sridhar, S. Kundu, S. K. Ghosh, A. Palla, A. Das, D. Crews, and D. A. Mathaikutty, "Llm-npu: Towards efficient foundation model inference on low-power neural processing units," in *2025 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*, Aug 2025, pp. 1–8.

[5] Intel Corporation, "Intel Meteor Lake: A Client Platform Built on the Intel 4 Process and Foveros Packaging (Hot Chips 2023)," 2023, conference presentation materials describing Meteor Lake tile-based architecture and packaging. [Online]. Available: https://hotchips.org/

[6] ——, "Heterogeneous AI Powerhouse: Unveiling the Hardware and Software Foundation of Intel Core Ultra Processors," 2024, official Intel Core Ultra architecture white paper including NPU (Intel AI Boost). [Online]. Available: https://cdrdv2-public.intel.com/817736/

[7] ——, "Intel Core Ultra Processor Product Brief (Meteor Lake)," 2023, intel product brief summarizing platform features, including NPU (Intel AI Boost) positioning and peak AI throughput claims.

[8] ——, "OpenVINO Documentation: NPU Devices, INT8 Inference, and Release Notes," 2025. [Online]. Available: https://docs.openvino.ai/

[9] Z. Bi, X. Chen, L. Sun, Y. Yao, Q. Shen, J. Lou, and C. Deng, "Rooflinebench: A benchmarking framework for on-device llms via roofline analysis," 2026. [Online]. Available: https://arxiv.org/abs/2602.11506

[10] C. Lam, "Intel Meteor Lake's NPU," Apr. 2024, technical analysis of Meteor Lake NPU internals; not peer-reviewed. [Online]. Available: https://chipsandcheese.com/p/intel-meteor-lakes-npu

[11] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "Dnnfusion: accelerating deep neural networks execution with advanced operator fusion," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 883–898. [Online]. Available: https://doi.org/10.1145/3453483.3454083

[12] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=F72ximsx7C1

[13] S. Kumar and S. Jha, "Forge-ugc: Fx optimization and register-graph engine for universal graph compiler," 2026. [Online]. Available: https://arxiv.org/abs/2604.16498

[14] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[15] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1810.00826

[16] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.02216

[17] J. Gasteiger, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1810.05997

[18] V. P. Dwivedi and X. Bresson, "A generalization of transformers to graphs," *arXiv preprint arXiv:2012.09699*, 2021. [Online]. Available: https://arxiv.org/abs/2012.09699

[19] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019. [Online]. Available: https://arxiv.org/abs/1908.08962

[20] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520. [Online]. Available: https://arxiv.org/abs/1801.04381

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: https://arxiv.org/abs/1512.03385

[22] R. Singh and S. S. Gill, "Edge ai: A survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667345223000196

[23] D. Xu, H. Zhang, L. Yang, R. Liu, G. Huang, M. Xu, and X. Liu, "Fast on-device llm inference with npus," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 445–462. [Online]. Available: https://doi.org/10.1145/3669940.3707239

[24] MLCommons, "MLPerf Inference Benchmark Suite (v4.1, v5.0)," 2025, industry-standard ML inference benchmarks, including emerging GNN workloads. [Online]. Available: https://mlcommons.org/benchmarks/inference/

[25] H. Shirzad, A. Velingker, B. Venkatachalam, D. J. Sutherland, and A. K. Sinop, "Exphormer: sparse transformers for graphs," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.